NIT Website API – Frontend Integration Guide

Nesma Infrastructure & Technology · Backend API v1 · For the Next.js website (https://nit-hazel.vercel.app)

1. Quick facts

Item	Value
Base URL (production)	`https://api.nesma-nit.com/api` (confirm the final domain before go-live)
Base URL (local)	`http://127.0.0.1:8000/api`
Authentication	None. No token, no cookies, no CSRF header
Format	JSON responses. Send JSON, or `multipart/form-data` for file uploads
Languages	English (default) and Arabic: add `?lang=ar`
Images	Always full `https://` URLs, ready for `<img>` / `next/image`
CORS	Allowed for `https://nit-hazel.vercel.app` and `http://localhost:3000`

Keep the base URL in an environment variable:

```
# .env.local
NEXT_PUBLIC_API_URL=https://api.nesma-nit.com/api
```

2. Endpoints at a glance

Method	Path	Used on
GET	`/blog-categories/`	Media centre tabs (NEWS / ARTICLE / ANNOUNCEMENT ...)
GET	`/blogs/`	Home "News & announcements", Media centre list
GET	`/blogs/{slug}/`	Media detail page `/media/{slug}` + "Similar news"
POST	`/enquiry/`	Footer form and Contact page form
GET	`/career-options/`	Careers form "Education" dropdown
POST	`/career-application/`	Careers form (with CV upload)

3. Language (English / Arabic)

- Every endpoint accepts `?lang=en` or `?lang=ar`. Without it the API reads the `Accept-Language` header, then falls back to English.
- For POST requests you can also send `lang` in the body.
- Text fields come back already translated. If the admin has not entered an Arabic value, the English value is returned, so you never get an empty title.
- Validation and success messages are returned in the requested language. Show `message` and `errors` exactly as received.
- The response header `Content-Language` tells you which language was served.

Tip: pass the active Next.js locale on every call, e.g. `/blogs/?lang=ar`.

4. Media centre (blogs)

4.1 GET /blog-categories/

Returns the active categories that contain at least one published post, in admin order. Use them to build the filter tabs. Not paginated.

```
[
  { "id": 1, "name": "News", "slug": "news" },
  { "id": 2, "name": "Article", "slug": "article" },
  { "id": 3, "name": "Announcement", "slug": "announcement" }
]
```

4.2 GET /blogs/

Query param	Type	Description
`lang`	`en` / `ar`	Response language
`category`	slug or id	Filter by category, e.g. `category=news`. `all` or empty = no filter
`search`	string	Search in English and Arabic titles
`exclude`	slug	Leave out one post (useful on detail pages)
`page`	number	Page number, starts at 1
`page_size`	number	Items per page. Default 9, maximum 50

```
{
  "count": 6,
  "total_pages": 1,
  "current_page": 1,
  "next_page": null,
  "previous_page": null,
  "results": [
    {
      "id": 12,
      "slug": "sar-160-million-contract",
      "title": "Nesma Infrastructure & Technology Signs a Contract Valued at Over SAR 160 Million",
      "category": { "id": 1, "name": "News", "slug": "news" },
      "location": "Riyadh, Saudi Arabia",
      "date": "2026-03-12",
      "formatted_date": "MAR 12, 2026",
      "image": "https://api.nesma-nit.com/media/blogs/s-1.png",
      "short_description": "We are proud to announce ..."
    }
  ]
}
```

- `next_page` and `previous_page` are page numbers (or `null`), not URLs. Pass them back as `?page=`.
- `formatted_date` matches the site style: `MAR 12, 2026` in English, Arabic month names with `lang=ar`. Use `date` (ISO) if you format dates yourself.
- Home page "News & announcements": `GET /blogs/?page_size=5`.

4.3 GET /blogs/{slug}/

Same fields as a list item, plus:

Field	Description
`content`	Article body as HTML (from the admin rich-text editor). Images inside it already use full https URLs
`meta_title`	SEO title (falls back to the title)

Field	Description
`meta_description`	SEO description (falls back to the short description)
`updated_at`	ISO timestamp of the last edit
`related`	Up to 3 other posts for "Similar news & announcements". Same category first, then newest

If the slug does not exist or the post is unpublished, the API returns `404`:

```
{ "status": "error", "message": "Blog not found." }
```

In Next.js call `notFound()` on a 404. Render `content` with `dangerouslySetInnerHTML`. The HTML is written by NIT admins in the admin panel.

5. Enquiry forms (footer + contact page)

POST /enquiry/

Both forms use the same endpoint. The `source` field tells the backend which form was used, and the admin panel shows it.

Footer form ("Join Our Growth Journey"):

```
{
  "source": "footer",
  "full_name": "Ahmed Ali",
  "email": "ahmed@example.com",
  "message": "I would like to know more about investing.",
  "website": ""
}
```

Contact page form ("Contact form"):

```
{
  "source": "contact",
  "first_name": "Sara",
  "last_name": "Khan",
  "company_name": "ACME Holdings",
  "phone": "+966 55 063 4599",
  "email": "sara@example.com",
  "message": "We need a quote for a substation project.",
  "website": ""
}
```

Field	Footer	Contact	Rules
`source`	`footer`	`contact`	Required
`full_name`	required	-	Max 300 characters
`first_name`	-	required	Max 150
`last_name`	-	required	Max 150
`company_name`	-	optional	Max 200
`phone`	-	optional	7-20 characters: digits, spaces, `+ - ( ) .`
`email`	required	required	Valid email
`message`	required	optional	Free text
`website`	hidden	hidden	Honeypot, always send empty (see section 8)

Every new enquiry is emailed to the NIT admin straight away, with reply-to set to the sender.

6. Careers form

6.1 GET /career-options/

Values for the "Education" dropdown. Show `label` and submit `value`.

```
{
  "education": [
    { "value": "High School", "label": "High School" },
    { "value": "Diploma", "label": "Diploma" },
    { "value": "Bachelor's Degree", "label": "Bachelor's Degree" },
    { "value": "Master's Degree", "label": "Master's Degree" },
    { "value": "PhD", "label": "PhD" }
  ]
}
```

With `lang=ar` the labels are in Arabic, and the values stay in English.

6.2 POST /career-application/

Send as `multipart/form-data` (it includes a file). Do not set the `Content-Type` header yourself. The browser adds the multipart boundary automatically.

Field	Required	Rules
`first_name`	yes	Max 150
`last_name`	no	Max 150
`email`	yes	Valid email
`mobile`	yes	7-20 characters: digits, spaces, `+ - ( ) .`
`current_location`	yes	Max 200
`nationality`	no	Free text, e.g. "Saudi Arabian" (the site's dropdown values are fine)
`education`	yes	One of the `value`s from `/career-options/`
`degree`	yes	Max 200
`cv`	yes	PDF, DOC or DOCX. Maximum 5 MB. The file contents are checked, not just the extension
`website`	hidden	Honeypot, always empty
`lang`	no	`en` / `ar` (or use `?lang=`)

Each application is emailed to the NIT admin with the CV attached.

7. Responses and errors

Status	Meaning	Body
200	OK (GET)	Data as shown above
201	Form submitted	`{ "status": "success", "message": "...", "data": { "id": 12 } }`
400	Validation failed	`{ "status": "error", "message": "...", "errors": { "field": ["..."] } }`
404	Not found	`{ "status": "error", "message": "..." }`
429	Too many submissions	`{ "detail": "Request was throttled. Expected available in 3540 seconds." }`
500	Server error	`{ "status": "error", "message": "Server error. Please try again later." }`

Validation error example (with `lang=en`):

```
{
  "status": "error",
  "message": "Please correct the errors below.",
  "errors": {
    "email": ["Enter a valid email address."],
    "cv": ["Only PDF, DOC or DOCX files are allowed."]
  }
}
```

Each key in `errors` is a form field name and holds a list of messages. Show the first message under the matching input, and show `message` as the form-level alert.

8. Spam protection

- Honeypot: add a hidden input named `website` to both forms and always send it empty. Hide it with CSS rather than `type="hidden"`, so bots still fill it in. If it has a value, the API returns a normal success response but saves nothing.
- Rate limit: each IP can submit 10 forms per hour (enquiry and career applications combined). Beyond that the API returns `429`. Show a friendly "please try again later" message.

9. Caching and performance

- Blog responses are cached on the server and refreshed instantly when an admin saves, publishes or deletes a post or category. There's no need to bust the cache from the frontend.
- GET responses include `Cache-Control: public, max-age=60`.
- Recommended in Next.js: `fetch(url, { next: { revalidate: 60 } })` for blog pages. Never cache POST requests.
- Images are optimised losslessly on upload (no quality change). Add the media domain to `next.config.js` so `next/image` can resize them:

```
// next.config.js
module.exports = {
  images: {
    remotePatterns: [{ protocol: 'https', hostname: 'api.nesma-nit.com', pathname: '/media/**' }],
  },
};
```

10. Next.js code examples

10.1 Small API helper

```
// lib/api.ts
const API = process.env.NEXT_PUBLIC_API_URL;

export async function getJSON<T>(path: string, locale: string, revalidate = 60): Promise<T | null> {
  const sep = path.includes('?') ? '&' : '?';
  const res = await fetch(`${API}${path}${sep}lang=${locale}`, { next: { revalidate } });
  if (res.status === 404) return null;
  if (!res.ok) throw new Error(`API ${res.status}`);
  return res.json();
}
```

10.2 Media centre list and tabs

```
const [categories, blogs] = await Promise.all([
  getJSON<Category[]>('/blog-categories/', locale),
  getJSON<BlogPage>(`/blogs/?category=${tab || 'all'}&page=${page}`, locale),
]);
```

10.3 Media detail page

```
// app/[locale]/media/[slug]/page.tsx
import { notFound } from 'next/navigation';

export default async function Page({ params: { locale, slug } }) {
  const blog = await getJSON<BlogDetail>(`/blogs/${slug}/`, locale);
  if (!blog) notFound();
  return <article dangerouslySetInnerHTML={{ __html: blog.content }} />;
}

export async function generateMetadata({ params: { locale, slug } }) {
  const blog = await getJSON<BlogDetail>(`/blogs/${slug}/`, locale);
  return blog ? { title: blog.meta_title, description: blog.meta_description } : {};
}
```

10.4 Footer / contact form (JSON)

```
async function submitEnquiry(values: Record<string, string>, locale: string) {
  const res = await fetch(`${API}/enquiry/?lang=${locale}`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ ...values, website: '' }),
  });
  const json = await res.json();
  if (res.status === 429) return { ok: false, message: 'Too many requests, please try later.' };
  return { ok: json.status === 'success', message: json.message, errors: json.errors ?? {} };
}
```

10.5 Careers form (multipart with CV)

```
async function submitApplication(form: HTMLFormElement, locale: string) {
  const data = new FormData(form); // includes the <input type="file" name="cv">
  const res = await fetch(`${API}/career-application/?lang=${locale}`, { method: 'POST', body: data });
  return res.json(); // { status, message, errors? }
}
```

10.6 TypeScript types

```
type Category = { id: number; name: string; slug: string };

type BlogItem = {
  id: number; slug: string; title: string; category: Category;
  location: string; date: string; formatted_date: string;
  image: string | null; short_description: string;
};

type BlogPage = {
  count: number; total_pages: number; current_page: number;
  next_page: number | null; previous_page: number | null; results: BlogItem[];
};

type BlogDetail = BlogItem & {
  content: string; meta_title: string; meta_description: string;
  updated_at: string; related: BlogItem[];
};
```

11. Integration checklist

- `NEXT_PUBLIC_API_URL` is set for each environment.
- Every request sends the active locale (`?lang=en|ar`).
- Media tabs come from `/blog-categories/`, not hard-coded.
- Detail pages use `slug` and call `notFound()` on 404.
- The media domain is added to `images.remotePatterns`.
- Both forms send `source`, plus the empty `website` honeypot.
- Errors are shown per field from `errors`, and `message` is the form alert.
- The careers form uses `FormData` and doesn't set `Content-Type` manually.
- 429 responses show a friendly retry message.